Введение в глубокое обучение

Глубокое обучение

Итак, мы начинаем основную тему этого семестра — глубокое обучение. Ранее мы рассматривали алгоритмы т.н. классического машинного обучения (classic ML), однако время и развитие технологий сделали своё время, и позволили открыть наиболее сильные решения для тех или иных доменов.

Определение 1 (Deep Learning) Глубокое обучение — это подмножество машинного обучения, которое в свою очередь является подмножеством ИИ. Основной сутью глубокого обучения является использование специальных архитектур, в которых наращивают «глубину» (количество параметров модели), что позволяет этим моделям распознавать наиболее сложные паттерны в данных и выполнять задачи, которые ещё недавно считались невозможными для компьютера.

> **Замечание**
>
> Однако это не означает, что classic ML неактуален на данный момент: как минимум методы оттуда используются для построения т.н. baseline решений (первых решений, результаты которого используются как отправная точка), а как максимум — ансамблевые методы всё равно зачастую справляются лучше на табличных данных за счёт того, что используют меньше весов.

Своего рода революция в машинном обучении произошла в начале 2000-х, когда Джеффри Хинтон и Йошуа Бенджи научились обучать глубокие сети, благодаря чему удалось решить задачу распознавания речи. С тех пор всё, что распознает голос и чем вы, возможно, привыкли пользоваться (Яндекс Алиса, Apple Siri и пр.), работает строго на основе искуственного интеллекта. Что интересно, до этого «прорыва» глубокое обучение считалось бесперспективной веткой машинного обучения, которую исследовали, грубо говоря, ради интереса.

Что позволило этой революции произойти?

1. Предобучение с учителем.

Примерно в то время уже говорили о том, что градиентный спуск на таких моделях работает очень плохо (мы ещё обязательно вернёмся к этому вопросу в будущем), что не позволяет их, собственно, обучать. Тогда предложили следующую парадигму, которая должна была это решать.

Определение 2 (Self-supervised Learning) Предобучение с учителем — техника, при которой модель самостоятельно генерирует обучающий сигнал (таргеты) из самой структуры данных. Это позволяет использовать колоссальные объемы неразмеченной информации и наделяет модель глубоким пониманием внутренних закономерностей данных еще до начала основной задачи, => такая модель требует меньше обучения в будущем.

2. Большое количество данных.

Со временем банально стало больше данных. Важно помнить, что много данных для модели не бывает, а мало — ещё как бывает. В этом контексте также важно Определение 2, которое позволяло не тратить колоссальное количество ресурсов и времени на разметку данных (на выставление значения целевой переменной).

3. Рост вычислительных мощностей.

Видеокарты и процессоры также меняются, что позволило оптимизировать высокозатратные операции (например, перемножение матриц). Все современные флагманские модели обучаются

на огромном кластере видеокарт, что позволяет достигать каких-то, казалось бы, немыслиммых результатов.

Граф вычислений, дифференциирование на нём.

Важно помнить, что по факту основная наша цель в машинном обучении — это построение функции. Функцию построить то можно какую угодно (главное, чтобы мы потом могли объяснить её смысл), однако здесь появляется другой вопрос. В цикле модели неизбежно есть этап оптимизации этой функции, соответственно, нам нужно понимать, дифференциируема ли наша функция.

Есть довольно простая мысль: если от каждой простой части функции можно взять производную, то производную получится взять и от исходной сложной функции. Соответственно, нам нужно некое представление, которое позволит представлять сложную функцию как набор более простых. Самый простой и наглядный способ это сделать — граф вычислений.

Определение 3 (Computation Graph) Граф вычислений — структура данных, которая представляет собой граф, в узлах которого находятся составные компоненты исходной функции, а рёбра представляют собой связанность каждой из компонент.

Вот пример разложения функции $f(x, y) = x^2 + xy + (x + y)^2$ тремя различными способами:

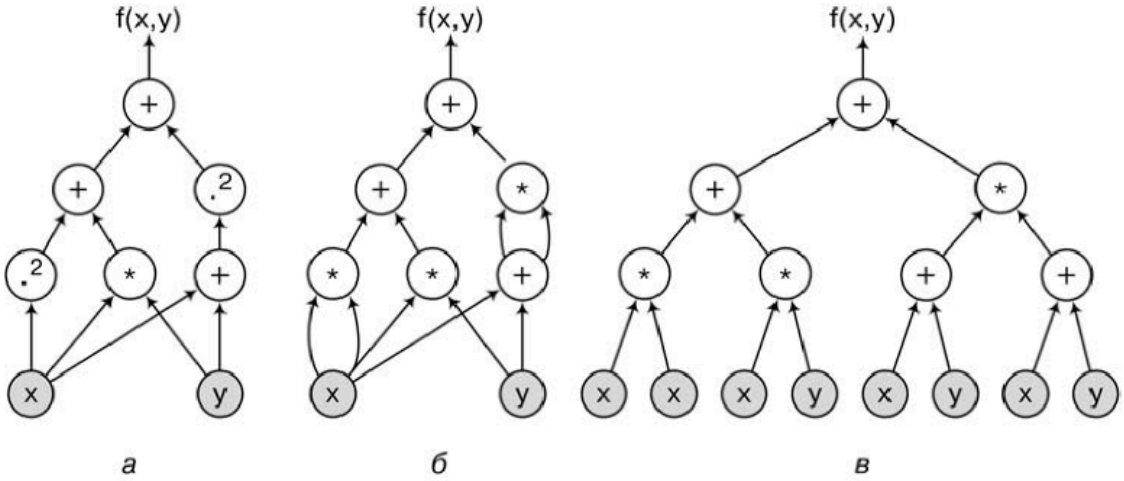


Рис. 1. Представление f как набор простых компонент.

Теперь мы знаем, что сколь угодно сложную функцию можно разложить в граф, и если от каждого узла можно найти производную, то производную можно посчитать и от исходной функции. Давайте вспомним ещё одно занимательное правило из математики:

$$(f \circ g)'(x) = (f(g(x)))' = f'(g(x))g'(x) \tag{1}$$

Из этого следует, что если f зависит от g, а g зависит от x, то маленькое изменение параметра x на δx приведет к маленькому изменению $g(x)$ примерно на $\delta g = g'(x)\delta x$. Оно же в свою очередь изменит $f(g(x))$ примерно на $\delta f = f'(g(x))\delta g = f'(g(x))g'(x)\delta x$

Единственный разумный шаг, который нам нужно добавить – функции могут быть векторными, т.е. принимать вектор в качестве аргумента. В таком случае речь идёт не о производной, а о **градиенте**[1], векторе частных производных:

$$\nabla_x(f \circ g) = \frac{\partial f}{\partial g} \nabla_x g \tag{2}$$

Если g также будет являться вектором, то выходит, что f зависит от x не в одном месте, а сразу в нескольких: $f = (g_1(x), g_2(x), ..., g_k(x))$. Значит, приращение δx превратится в несколько разных приращений вида $\delta g_1, ..., \delta g_k$, каждое из которых внесёт разный вклад в приращение функции f, т.е.:

$$\delta f = \frac{\partial f}{\partial g_1} \delta g_1 + ... + \frac{\partial f}{\partial g_k} \delta g_k \tag{3}$$

В конечном итоге, по графу вычислений можно считать производную снизу вверх (см. рис. б ниже), а можно считать сверху вниз (см. рис. в ниже). Спойлер, нас интересует второй вариант: представьте, что мы хотим посчитать градиент по функции потерь, и **распространить** его назад, чтобы понять, насколько стоит изменить веса.

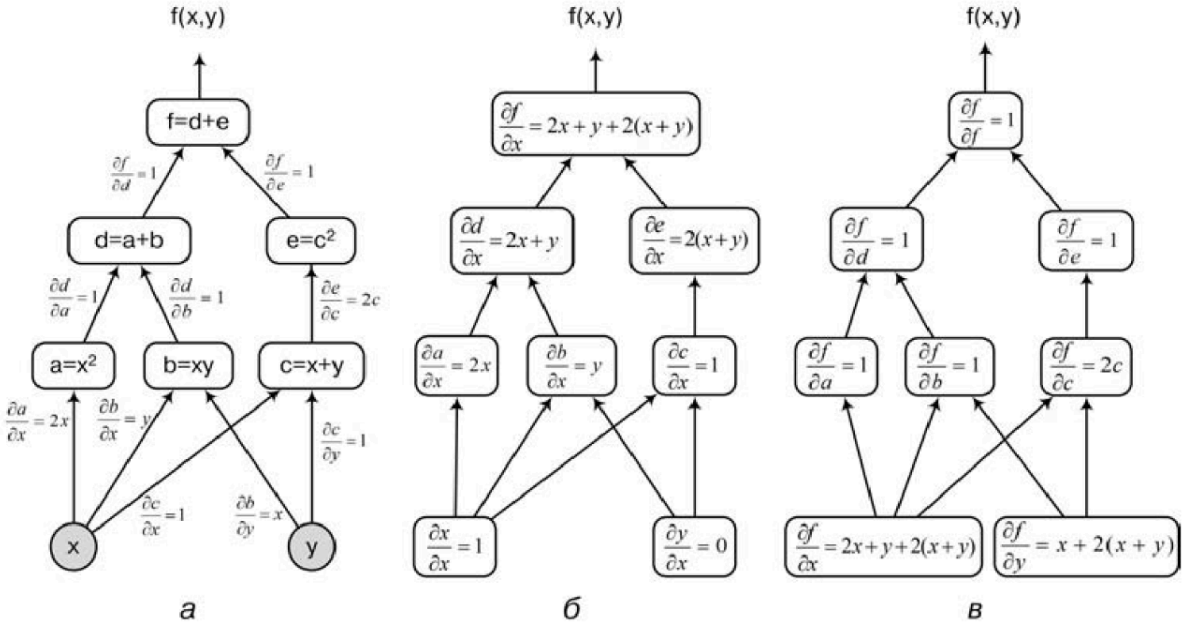


Рис. 2. Распространение градиента. Для первого ознакомления это довольно сложный ментальный шаг, поэтому советую тут задержаться.

Интересующий нас алгоритм, представленный третьим графом, называется **обратным распространением ошибки** (backprop, backpropagation).

> **Замечание**
>
> Backprop, в своей сущности, является комбинированием графа вычислений и градиентного спуска. Он довольно сложен для восприятия, но такой механизм в конечном итоге позволяет обучать сколь угодно глубокие модели, которые удовлетворяют такому условию: функция представима в виде графа, каждый узел которого дифференциируем.

[1]В действительности да — мы снова пришли к gradient descent, но на этот раз взглянули на проблему под другим углом

Ещё одно небольшое открытие: возможно вы краем уха, но слышали про такие библиотеки, как PyTorch, Tensorflow, JAX. Они позволяют делать какие-то невероятные вещи, и с помощью невероятно высокого уровня абстракции (спасибо Python) конструировать довольно сложные модели. Так вот: всё, что они в действительности делают, так это автоматизируют построение и дифференцирование графа вычислений[2], никакой магии откровенно не происходит.

[2]Если вам вдруг стало интересно почитать подробнее про это в рамках PyTorch, то вам сюда. Там и очень кратко, но напоминают суть градиентного спуска в целом.

Перцептрон Розенблатта

Мы всё ближе к конструированию нашей первой нейронной сети. Перед этим давайте ещё рассмотрим простейший кирпичик, из которого будет строиться всё в будущем: **перцептрон Розенблатта**[3].

На протяжении всей истории нейронных сетей (и по сей день), большая часть вдохновения для их развития становились уже известные факты из нейробиологии. К слову, я думаю это логично, если вдуматься в название «нейронные сети». Так вот, чтобы воплотить в жизнь активацию нейронов, была предложена следующая модель:

$$sign(w^T x) = sign(w_0 + w_1 x_1 + w_2 x_2 + ... + w_d x_d) \tag{4}$$

Особо внимательные слушатели и читатели лекций помнят, что такая модель была первым приближением в решении задачи бинарной классификации. Мы также обращали внимание, что она плохая, однако это было давно, поэтому предлагаю освежить память:

1. Не очень понятно, как брать производную от этой функции (особенно в нуле);
2. Такая модель в состоянии решать только линейно-разделимый случай, что встречается довольно редко.

Значит, мы хотим (а) перейти к более гладкой функции и (б) научить модель не гарантировать, а **предсказывать**.

Для этого мы применяли т.н. **сигмоиду**:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{5}$$

Такая функция отображала $\mathbb{R}$ в $[0, 1]$, а уже эти выходы мы воспринимали (научнее говорить интерпретировали) как вероятность положительного класса. Здесь важно понимать, что функций $\mathbb{R} \to [0, 1]$ (да и в сущности $\mathbb{R}$ в любое нетривиальное пространство) существует достаточно много помимо сигмоиды. Такие функции мы будем называть **функциями активации**.

Определение 4 (Activation Function) Функцией активации будем называть любую функцию, которую мы накладываем на выход одного перцептрона (т.е. применяем после подсчёта $w^T x$). Это позволит переходить к нелинейности, а значит, рассматривать более сложные и нетривиальные паттерны в данных.

Замечание

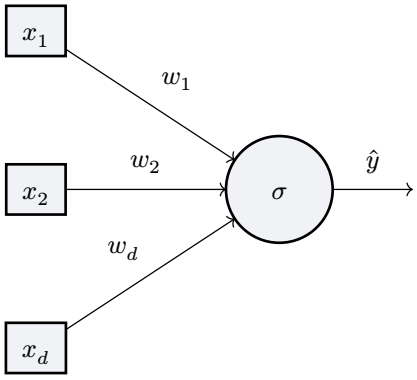
Функций активаций довольно много, однако основной ранее была именно сигмоида: она удобная, её применение прямиком исходит из метода максимального правдоподобия. Этот метод мы не рассматривали в рамках лекций, однако он лежит в основе многих приложений: разумность MSE в регрессии, почему логистическая регрессия умеет предсказывать вероятности и это разумное требование от неё, и пр. Это довольно большая тема, поэтому желающие могут почитать про это отдельно.

В дальнейшем модель вида вход $\to$ линейная комбинация $\to$ функция активации мы будем использовать везде, для конструирования наиболее сложных и способных моделей.

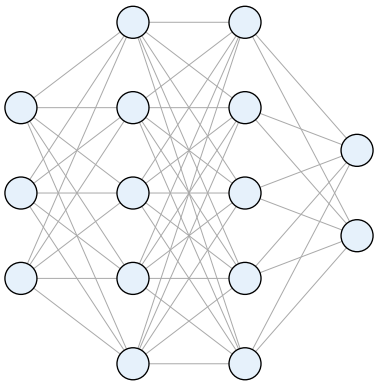
[3] у Николенко в книге «Глубокое обучение. Погружение в мир нейронных сетей» можно почитать подробнее про историю перцептрона

MultiLayer Perceptron

Итак, для наглядности:



Это – **логистическая регрессия**



А это – **многослойный перцептрон** (MultiLayer Perceptron)

Первый слой называется входным (input layer), последний слой называется выходным (output layer). Всё что между ними — скрытые слои (hidden state). Такая терминология будет применима вообще ко всем архитектурам в дальнейшем, поэтому лучше её запомнить сразу.

В общем то — это вся архитектура. Да, вот так просто мы перешли к многослойным моделям. Однако нам остался ещё один штрих — векторизировать нужные операции.

Если в слое k нейронов, и веса у них $w_1, w_2, ..., w_k$, где $w_i = (w_{i1}, w_{i2}, ..., w_{in})^T$, а на вход подаётся вектор $x = (x_1, x_2, ..., x_n)^T$, то в результате у нейрона с весами w_i мы получим выход $y_i = f(w_i^T x)$, где f – его функция активации.

Мы можем представить вычисление, которое делают все нейроны сразу, вот так:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_k \end{pmatrix} = y = f(Wx) = \begin{pmatrix} f(w_1^T x) \\ \vdots \\ f(w_k^T x) \end{pmatrix}, \text{ где } W = \begin{pmatrix} w_1 \\ \vdots \\ w_k \end{pmatrix} = \begin{pmatrix} w_{11} & ... & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{k1} & ... & w_{kn} \end{pmatrix} \tag{6}$$

и вычисление всего слоя сведётся к умножению матрицы весов на вектор входов, а затем покомпонетному применению одной и той же функции активации. Это оказывается сильно эффективнее ещё за счёт того, что видеокарты способны оптимизировать и кратно ускорить процесс такого умножения, что уже упоминалось ранее.

Функции активации

Sigmoid

Исторически было принято «активировать» нейрон с помощью сигмоиды:

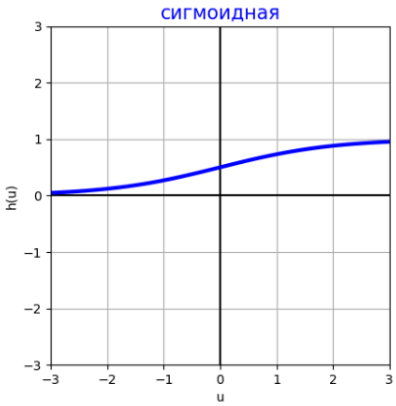
$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{7}$$

она ограничена, стремится к 0 при $x \to -\infty$ и к единице при $x \to \infty$.

Её производную легко посчитать как $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, доказывать это мы конечно же не будем.

При больших (или очень маленьких) значениях x производная сигмоиды практически равна нулю. В глубоких сетях, когда мы перемножаем эти маленькие производные , градиент становится нулём до того, как дойдет до первых слоев $\Rightarrow$ НС никак не будет обучаться.

$$h(u) = \sigma(u) = \frac{1}{1 + e^{-u}}$$



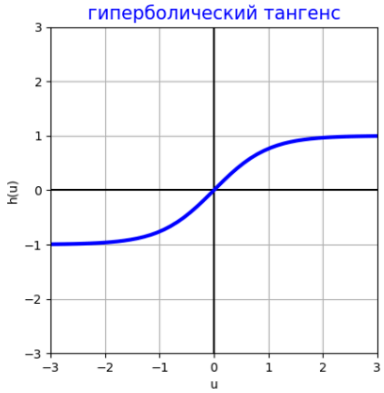
Tanh

Сигмоида имеет насыщение в точке 0, т.е. если пытаться обучить значение этой функции в ноль, вход будет стремиться к $-\infty$, а производная к нулю, что затрудняет градиентный спуск. Частично решить эту проблему призван гиперболический тангенс:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{8}$$

Для сравнения: $\tanh'(0) = 1,\ \text{а}\ \sigma'(0) = \frac{1}{4}$. Справедливости ради, проблема затухания на хвостах по прежнему остаётся, но теперь она не такая критическая.

$$h(u) = \text{tangh}(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}} = 2\sigma(2u) - 1$$



SoftSign

Однако tanh по-прежнему быстро выходит на асимптоты $+-1$, что может испортить градиентный спуск.

$$SF(x) = \frac{x}{1 + |x|} \tag{9}$$

SoftSign идейно повторяет tanh, но имеет характер приближения к асимптотам +1 и −1 полиномиальный, а не экспоненциальный, вследствие чего на значения $+-1$ она выходит медленнее, что ускоряет сходимость при настройке сети.

$$h(u) = \frac{u}{1 + |u|}$$

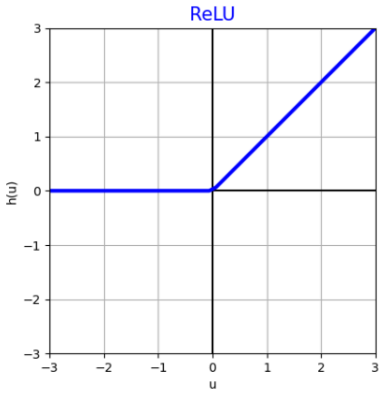
Rectified Linear Units (ReLU)

$$ReLU(x) = \max(0, x) \tag{10}$$

Имеет нетривиальный градиент +1 на полуоси $x > 0$, за счёт чего используется как в скрытых слоях, так и в выходных слоях, где нужно предсказывать неотрицательный регрессионный отклик.

Dead ReLU: Если на нейрон прилетает большой отрицательный градиент, веса могут обновиться так, что он всегда будет выдавать 0 для любых входов. Такой нейрон «умирает» и больше никогда не обновится, так как его градиент в этой области – чистый ноль.

$$h(u) = \max\{0; u\}$$



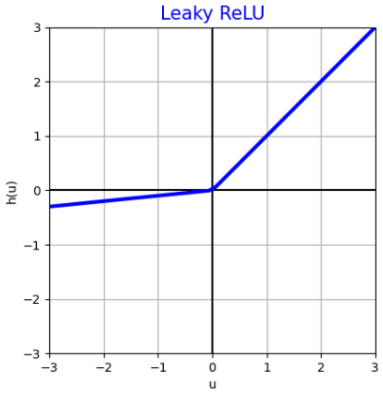
Leaky ReLU

Именно для решения этой проблемы придумали Leaky ReLU, который исключает холостые вычисления:

$$h(x) = \max(\alpha x, x), \alpha \in (0, 1) \tag{11}$$

Активация Leaky ReLU обладает всеми достоинствами ReLU-активации, но кроме того выдаёт ненулевые значения и для отрицательных аргументов, за счёт чего не вырождается в ноль и не приводит к лишним вычислениям вхолостую.

$$h(u) = \max\{\alpha u; u\},\ \alpha \in (0, 1) - \text{гиперпараметр}$$



ReLU и Leaky ReLU являются наиболее предпочтительными функциями активации.